

Food Waste Manager

Styczeń 2026 rok

Magdalena Kochanowska, s35136

Dawid Dzierżęcki, s36683

Lena Marusik, s25273

1. Wprowadzenie	3
1.1. Problem biznesowy	3
1.2. Wizja systemu	3
1.3. Podsumowanie zastosowanych technologii	3
2. Opis architektury systemu	4
2.1. Schemat architektury (graf przepływu danych i komponentów)	4
2.2. Źródła danych.	4
2.3. Opis poszczególnych modułów	6
2.3.1. Ingestion (CV PDF -> knowledge graph)	6
2.3.2. RFP Parsing & Availability	6
2.3.3. Matching Engine	7
2.3.4. Conversation Agent (schemat grafu, jeśli został zaprojektowany, lista i opis tooli)	8
2.3.5. Frontend Layer	11
3. Knowledge Graph	13
3.1. Schemat grafu (encje i relacje).	13
3.2. Jak przetwarzano dane wejściowe: CV, RFP, pliki YAML/JSON.	15
3.3. Metody walidacji danych i zapewnienia jakości ekstrakcji.	17
4. Algorytm dopasowania danych z różnych źródeł (Matching Algorithm)	17
4.1. Jak działa algorytm dopasowania – krok po kroku.	18
4.2. Jak uwzględnia różne czynniki: skills match, doświadczenie, dostępność.	19
4.3. Wyjaśnienie sposobu punktacji / rankingowania kandydatów.	19
5. Business Intelligence / Zaawansowane zapytania	20
5.1. Lista obsługiwanych typów zapytań	20
5.1. Liczenie (Counting)	20
5.2. Filtrowanie (Filtering)	20
5.3. Agregacja (Aggregation)	20
5.4. Reasoning / Multi-hop reasoning	21
5.5. Zapytania czasowe (Temporal queries)	21
5.6. Złożone scenariusze biznesowe (What-if / optymalizacja)	21
5.2. Przykłady zapytań i odpowiedzi systemu	22
6. Wyniki eksperymentów / metryki	24
6.1. Dokładność dopasowania (matching accuracy)	24

6.2. Czas odpowiedzi na zapytania.	24
6.3. Efektywność grafu (średni czas przeszukiwania Cypher).	25
6.4. Porównanie GraphRAG vs. tradycyjny RAG:	26
6.4.1. Porównanie wydajności	26
6.4.2. Dokładność odpowiedzi na złożone zapytania	26
6.4.3. Czas odpowiedzi	26
6.4.4. Wybrane metryki	26
7. Wnioski i rekomendacje	27
7.1. Co działało najlepiej w systemie GraphRAG?	27
7.2. Gdzie tradycyjny RAG był niewystarczający?	27
7.3. Ewentualne ograniczenia i kierunki rozwoju (dynamiczne RFP, real-time updates)?	28
7.4. Czego nie udało się zrobić?	28
7.5. Jak system ma ograniczenia?	29
8. Załączniki / dokumentacja techniczna	29
8.1. Instrukcje uruchomienia systemu (Docker, Neo4j, środowisko Python).	29

Podział prac:

Frontend – Lena Marusik

Backend – Magdalena Kochanowska

Database + Backend ORM – Dawid Dzierżęcki

Poprawki + Dokumentacja – Lena+ Magdalena + Dawid

1. Wprowadzenie

1.1. Problem biznesowy

Problemem biznesowym, na który odpowiada projekt "Food Waste Manager", jest marnowanie żywności w gospodarstwach domowych, wynikające z braku wiedzy o możliwościach wykorzystania posiadanych produktów oraz ich terminach przydatności do spożycia. Użytkownicy często nie są świadomi, jakie potrawy mogą przygotować z dostępnych składników lub jak zaplanować posiłki w sposób efektywny czasowo i dietetycznie. System ma na celu inteligentne wspieranie decyzji kulinarnych, minimalizację strat żywności oraz optymalizację zakupów spożywczych.

1.2. Wizja systemu

Food Waste Manager to inteligentna aplikacja wspierająca użytkownika w planowaniu posiłków w oparciu o posiadane produkty, przepisy kulinarne oraz indywidualne preferencje żywieniowe. System umożliwia interakcję w formie chatu, pozwalając na zadawanie złożonych pytań, takich jak dopasowanie dań do ograniczeń dietetycznych, dostępnego czasu czy produktów z krótką datą ważności. Aplikacja wykorzystuje grafową reprezentację danych, aby łączyć przepisy, składniki i kontekst użytkownika, oferując spersonalizowane i praktyczne rekomendacje kulinarne.

1.3. Podsumowanie zastosowanych technologii

Backend aplikacji został zaimplementowany w języku **Python 3.12** i uruchamiany w środowisku **Docker**, z wykorzystaniem **Docker Compose** do orkiestracji usług.

System opiera się na **grafowej bazie danych Neo4j**, która przechowuje przepisy, składniki oraz ich relacje, a także wektorowe reprezentacje tekstu wykorzystywane do wyszukiwania semantycznego.

Logika przetwarzania języka naturalnego została zbudowana w oparciu o bibliotekę **LangChain**, integrującą modele **Azure OpenAI** zarówno do ekstrakcji ustrukturyzowanych danych z tekstu (LLM), jak i do generowania **embeddingów wektorowych**.

Dzięki zastosowaniu indeksów wektorowych w Neo4j możliwe jest semantyczne wyszukiwanie fragmentów przepisów w architekturze typu **RAG (Retrieval-Augmented Generation)**.

Warstwa prezentacji została zrealizowana jako **lekki frontend w Streamlit**, pełniący rolę interaktywnego interfejsu użytkownika do zadawania zapytań tekstowych i prezentowania wygenerowanych odpowiedzi.

Aplikacja realizuje kompletny pipeline ingestion danych: od analizy tekstu przepisu przez model językowy, przez **chunking i embeddingi**, aż po zapis do grafowej bazy danych wraz z relacjami domenowymi.

Takie połączenie technologii umożliwia elastyczne modelowanie wiedzy kulinarnej oraz budowę inteligentnego backendu pod asystenta kuchennego i system rekomendacji ograniczający marnowanie żywności.

2. Opis architektury systemu

2.1. Schemat architektury systemu

System został zaprojektowany w architekturze modułowej, warstwowej, łączącej:

- LLM-based reasoning (Conversation Agent)
- Graph-based retrieval (Neo4j / GraphRAG)
- Classic RAG (vector search over chunks)

Główne komponenty:

1. Frontend / Client
2. API Gateway (FastAPI)
3. Conversation Agent
4. Tool Layer (AgentTools)
5. Ingestion Pipeline
6. Matching Engine
7. Database Layer (Neo4j + Vector Index)
8. LLM Client (Azure OpenAI)

Przepływ danych:

1. Użytkownik wysyła zapytanie (tekst / URL / PDF / obraz).
2. API przekazuje żądanie do Conversation Agent.
3. Agent:
 - a. interpretuje intencję,
 - b. wybiera jedno narzędzie (tool),
 - c. przekazuje argumenty.
4. Wybrane narzędzie:
 - a. wykonuje Graph Search, RAG, Ingest, Matching lub Pantry logic.
5. Warstwa bazy danych:
 - a. Neo4j obsługuje graf (przepisy, składniki, relacje),
 - b. indeks wektorowy obsługuje semantyczne wyszukiwanie chunków.
 - c. wynik wraca do użytkownika w ustrukturyzowanej formie.

2.2. Źródła danych.

System Food Waste Manager wykorzystuje kilka komplementarnych źródeł danych, które wspólnie tworzą spójny graf wiedzy wykorzystywany do zaawansowanego wnioskowania i

odpowiadania na zapytania użytkownika. Dane te obejmują zarówno dane statyczne, jak i dynamicznie dostarczane przez użytkownika w trakcie interakcji z systemem.

1. Baza przepisów kulinarnych (dataset zewnętrzny – Kaggle)

Podstawowym źródłem danych dla systemu jest zbiór przepisów kulinarnych pochodzący z publicznego datasetu udostępnionego na platformie Kaggle: Recipe Dataset <https://www.kaggle.com/datasets/wilmerarltstrmberg/recipe-dataset-over-2m/data>

Na potrzeby projektu wykorzystano około 200 przepisów, które stanowią bazę wiedzy systemu. Dane te obejmują m.in.:

- nazwę przepisu,
- listę składników,
- instrukcje przygotowania,
- dodatkowe informacje opisowe (np. czas przygotowania).

Przepisy są przetwarzane i zapisywane w grafowej bazie danych Neo4J, gdzie reprezentowane są jako węzły połączone relacjami ze składnikami oraz innymi cechami opisowymi, co umożliwia efektywne filtrowanie i reasoning.

2. Dane o produktach spożywczych

Drugim istotnym źródłem danych są informacje o produktach spożywczych wykorzystywanych w przepisach. Produkty te zostały:

- wyekstraktowane na podstawie analizy częstotliwości występowania składników w bazie przepisów,
- ograniczone do zbioru 20–30 najczęściej występujących produktów, co pozwala uprościć model danych i poprawić wydajność zapytań.

Produkty są powiązane z przepisami relacjami w grafie wiedzy, co umożliwia:

- sprawdzanie dostępności składników,
- generowanie list brakujących produktów,
- optymalizację wykorzystania zapasów domowych.

3. Dane wprowadzane przez użytkownika (input dynamiczny)

System umożliwia dynamiczne rozszerzanie bazy danych poprzez dane dostarczane przez użytkownika w trakcie korzystania z aplikacji:

a) Przepisy dodawane przez użytkownika

Użytkownik może dodać nowe przepisy do systemu w jednej z trzech form:

- opis słowny w ramach chatu z aplikacją,
- link do strony HTML zawierającej przepis,
- zdjęcie lub zrzut ekranu przepisu (PDF / image).

Dostarczone dane są analizowane przez komponenty NLP, a następnie integrowane z grafową bazą danych.

b) Produkty posiadane w domu

Użytkownik może określić produkty, które aktualnie posiada poprzez wpisanie ich w formie tekstowej w chacie.

Dane te stanowią kontekst dla zapytań i umożliwiają personalizację rekomendacji.

4. Dane kontekstowe rozmowy

Dodatkowym źródłem danych są informacje wynikające z historii rozmowy użytkownika z systemem. Kontekst konwersacji umożliwia:

- wieloetapowe zapytania (multi-hop reasoning),
- interpretację złożonych scenariuszy decyzyjnych,
- zachowanie spójności dialogu.

Podsumowanie

Źródła danych w systemie Food Waste Manager łączą publiczny dataset przepisów z Kaggle z danymi produktowymi oraz dynamicznym inputem użytkownika. Połączenie tych danych w grafowej bazie Neo4J stanowi podstawę do realizacji zaawansowanych zapytań analitycznych oraz scenariuszy minimalizujących marnowanie żywności.

2.3. Opis poszczególnych modułów

2.3.1. Ingestion (Recipe PDF/ IMAGE/ URL -> knowledge graph)

Moduł **Ingestion** odpowiada za pozyskiwanie danych wejściowych w postaci tekstu, plików PDF, obrazów oraz adresów URL i ich normalizację do postaci surowego tekstu. W zależności od typu źródła wykorzystywane są dedykowane adaptory: parser HTML dla URL, ekstrakcja tekstu i tabel z PDF oraz OCR dla obrazów. Uzyskany tekst wejściowy przekazywany jest do warstwy LLM, która odpowiada za parsowanie treści przepisu do obiektów domenowych, takich jak Recipe i Ingredient. Następnie dane są dzielone na mniejsze fragmenty (chunking), co umożliwia efektywne osadzanie wektorowe oraz dalsze przetwarzanie semantyczne. Każdy fragment tekstu wzbogacany jest o embeddingi oraz powiązania z wykrytymi składnikami. Moduł buduje strukturę Document–Chunk–Recipe, która stanowi podstawę grafu wiedzy. Dane zapisywane są w bazie grafowej wraz z relacjami pomiędzy encjami. Ingestion obsługuje również ingestowanie użytkowników oraz ich zasobów spiżarni. W tym celu wykorzystywany jest model językowy do dopasowywania składników do istniejących encji w grafie. Cały proces zapewnia spójne i ustandaryzowane zasilanie systemu w dane niezależnie od formatu wejściowego.

2.3.2. RFP Parsing & Availability

Moduł odpowiada za wczytywanie, parsowanie i strukturyzację przepisów z różnych źródeł (tekst, URL, PDF, zdjęcie) oraz za ich zapisywanie w grafowej bazie Neo4j w sposób idempotentny.

Główne odpowiedzialności skoncentrowane są w dwóch klasach:

- **IngestionService**
 - pobiera i normalizuje surowy tekst z dowolnego źródła (pdfplumber, BeautifulSoup, EasyOCR)
 - uruchamia proces cały proces zaczynając od parsowania kończąc na wprowadzeniu danych do bazy
- **LLMClient**
 - wykonuje ekstrakcję i normalizację struktury przepisu
 - inferuje profile dietetyczne, tagi, sezonowość, kategorie składników
 - dopasowuje semantycznie nowe nazwy składników do istniejących w bazie

Cała logika parsowania, wnioskowania i dopasowywania nazw jest wyodrębniona do LLMClient, a IngestionService pełni rolę orkiestratora

2.3.3. Matching Engine

Matching Engine jest komponentem architektury systemu odpowiedzialnym za spójne dopasowanie oraz normalizację danych pochodzących z różnych źródeł wejściowych przed ich zapisaniem w bazie grafowej. Jego głównym celem jest zapewnienie jednoznacznej reprezentacji pojęć w grafie wiedzy, w szczególności w zakresie encji typu Ingredient, oraz eliminacja redundancji wynikającej z wielokrotnego występowania tych samych składników pod różnymi nazwami.

Komponent ten realizuje proces mapowania nowych danych na istniejące encje w grafie, co pozwala na uniknięcie duplikatów oraz zachowanie spójności semantycznej całej bazy danych. Matching Engine pełni rolę warstwy integracyjnej pomiędzy danymi użytkownika (takimi jak zawartość spiżarni), danymi pozyskiwanymi z zewnętrznych źródeł (przepisy w formie tekstu, stron WWW, dokumentów PDF oraz obrazów przetwarzanych za pomocą OCR) oraz istniejącą strukturą grafu, która stanowi referencyjne źródło wiedzy systemu.

Dzięki zastosowaniu mechanizmu dopasowania semantycznego system jest w stanie agregować dane z wielu kanałów wejściowych w jednolitym modelu grafowym, bez naruszania integralności logicznej relacji pomiędzy encjami.

Matching Engine działa głównie w ramach procesu ingestion danych i jest uruchamiany w momencie dodawania nowych składników do systemu. Logika dopasowania została zaimplementowana przede wszystkim w metodzie `IngestionService.ingest_pantry_items_for_user`, która odpowiada za przetwarzanie danych wejściowych użytkownika oraz ich powiązanie z odpowiednimi encjami w bazie grafowej. W tym procesie wykorzystywana jest również funkcja `LLMClient.match_ingredients_to_existing`, realizująca semantyczne porównanie nowych nazw składników z listą składników już istniejących w systemie.

Zakres działania Matching Engine obejmuje obsługę niejednoznaczności wynikających z różnorodności danych wejściowych. Komponent uwzględnia różnice językowe, umożliwiając poprawne dopasowanie nazw składników zapisanych w różnych językach. Obsługuje również odmiany gramatyczne, warianty fleksyjne oraz synonimy kulinarne, które w naturalnym języku mogą odnosić się do tego samego produktu. Dodatkowo mechanizm ten został zaprojektowany z uwzględnieniem danych pochodzących z systemów OCR, gdzie możliwe są błędy rozpoznawania tekstu oraz nieprecyzyjne zapisy nazw składników.

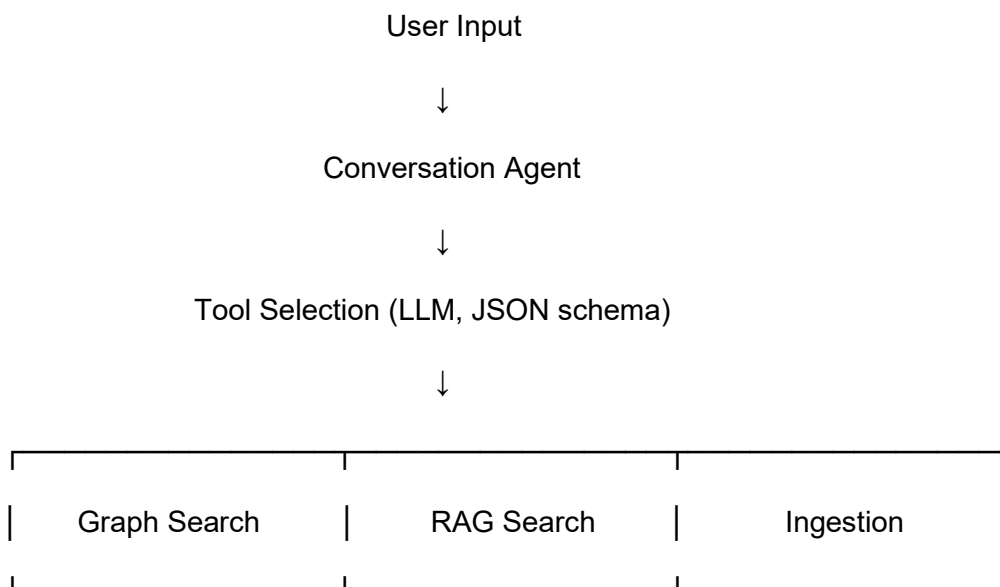
W przypadku braku jednoznacznego dopasowania lub niskiego poziomu pewności, Matching Engine inicjuje utworzenie nowej encji w grafie, co pozwala na zachowanie kompletności danych przy jednoczesnym ograniczeniu ryzyka błędnych połączeń semantycznych.

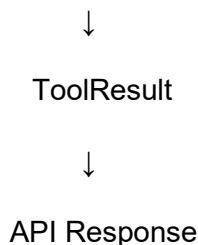
2.3.4. Conversation Agent (schmat grafu, jeśli został zaprojektowany, lista i opis tooli)

Conversation Agent jest centralnym komponentem warstwy orkiestracji systemu i pełni funkcję routera decyzyjnego pomiędzy intencją użytkownika, logiką biznesową oraz warstwą danych. Jego zadaniem nie jest generowanie odpowiedzi w języku naturalnym, lecz jednoznaczne mapowanie zapytań użytkownika na konkretne operacje systemowe realizowane przez wyspecjalizowane narzędzia (tools).

Agent analizuje treść wejściową zapytania, dostępny kontekst (np. identyfikator użytkownika, załączniki, adres URL) oraz stan systemu, a następnie wybiera dokładnie jedno narzędzie, które najlepiej odpowiada wykrytej intencji. Proces ten odbywa się z wykorzystaniem modelu językowego działającego w trybie strukturalnym, którego wyjściem jest obiekt JSON zgodny z predefiniowanym schematem.

Po wyborze narzędzia Conversation Agent przekazuje do niego ustrukturyzowane argumenty, inicjuje jego wykonanie oraz odbiera wynik w postaci obiektu ToolResult. Następnie wynik ten jest zwracany do warstwy API bez dodatkowej interpretacji lub modyfikacji treści. Takie podejście zapewnia jednoznaczny podział odpowiedzialności pomiędzy komponent decyzyjny a logikę wykonawczą oraz upraszcza dalszą rozbudowę systemu o nowe narzędzia.





Narzędzia ingestujące (Cel: pozyskanie i normalizacja przepisów z różnych źródeł):

ingest_recipe_text - Narzędzie służy do ingestii przepisu przekazanego w postaci surowego tekstu. Tekst wejściowy jest analizowany przez model językowy w celu ekstrakcji struktury przepisu, w tym listy składników, instrukcji, profili dietetycznych, tagów oraz czasu przygotowania. Następnie przepis zapisywany jest w bazie grafowej jako encja Recipe, a powiązany dokument źródłowy dzielony jest na fragmenty (chunki), dla których generowane są embeddingi wektorowe. Narzędzie wykorzystywane jest głównie w przypadku ręcznego wklejania treści przepisu.

ingest_recipe_url - Narzędzie realizuje ingest przepisu na podstawie adresu URL. W pierwszym kroku pobierana jest zawartość strony internetowej, z której usuwane są elementy nieistotne (np. skrypty, style, nagłówki nawigacyjne). Wyekstrahowany tekst jest następnie przetwarzany analogicznie jak w przypadku ingestii tekstowej: parsowany do postaci strukturalnej, zapisywany w grafie oraz indeksowany w warstwie RAG poprzez utworzenie chunków i embeddingów. Narzędzie umożliwia integrację przepisów z zewnętrжных serwisów kulinarnych.

ingest_recipe_pdf - Narzędzie umożliwia ingest przepisów zapisanych w formacie PDF. Obsługuje zarówno pliki przekazywane jako bajty binarne, jak i pliki dostępne lokalnie pod wskazaną ścieżką. Treść dokumentu jest ekstrakowana przy użyciu parsera PDF, obejmując zarówno tekst główny, jak i dane tabelaryczne. Następnie treść podlega dalszemu przetwarzaniu: parsowaniu do modelu domenowego, zapisowi w grafie oraz indeksacji semantycznej. Narzędzie przeznaczone jest do obsługi zeskanowanych książek kucharskich oraz dokumentów recepturowych.

ingest_recipe_image - Narzędzie realizuje ingest przepisów zapisanych w postaci obrazu. Wykorzystuje mechanizmy OCR do rozpoznania tekstu z obrazu, a następnie przetwarza uzyskany tekst w identyczny sposób jak inne źródła ingestii. Narzędzie obsługuje zarówno obrazy przesyłane jako pliki binarne, jak i obrazy dostępne lokalnie. Jest przeznaczone do digitalizacji przepisów pochodzących z fotografii, skanów lub zdjęć wykonanych urządzeniami mobilnymi.

Narzędzia wyszukiujące (Graph-based) (Cel: wyszukiwanie w grafie Neo4j):

search_recipes - Podstawowe narzędzie wyszukiwania przepisów w grafie Neo4j. Umożliwia filtrowanie wyników na podstawie wielu kryteriów, takich jak czas przygotowania, profile dietetyczne, wykluczone tagi, składniki uwzględniane w zapytaniu, kurs dania oraz ograniczenia wynikające ze spiżarni użytkownika. Wyszukiwanie realizowane jest deterministycznie przy użyciu zapytań Cypher.

search_recipes_from_pantry - Narzędzie realizuje wyszukiwanie przepisów możliwych do przygotowania na podstawie zawartości spiżarni użytkownika. Uwzględniane są wyłącznie składniki aktualnie posiadane przez użytkownika, przy czym system

automatycznie filtruje produkty o zerowej ilości. Narzędzie stanowi wyspecjalizowany wariant `search_recipes` zoptymalizowany pod scenariusze „gotowania z tego, co jest”.

search_recipes_from_list - Narzędzie umożliwia wyszukiwanie przepisów na podstawie jawnie podanej listy składników. Lista ta traktowana jest jako zbiór składników wejściowych niezależny od zawartości spiżarni użytkownika. Narzędzie pozwala zarówno na częściowe, jak i pełne dopasowanie składników w zależności od konfiguracji zapytania.

search_recipes_expiring - Narzędzie służy do wyszukiwania przepisów wykorzystujących produkty zbliżające się do daty ważności w spiżarni użytkownika. Uwzględnia ono zakres czasowy określony w dniach oraz filtruje produkty o dodatniej ilości. Mechanizm ten wspiera redukcję marnowania żywności poprzez priorytetyzację składników wymagających szybkiego zużycia.

search_recipes_under_time - Narzędzie umożliwia wyszukiwanie przepisów, których czas przygotowania nie przekracza określonego limitu. Wyszukiwanie realizowane jest bez konieczności dopasowania składników, co pozwala na szybkie uzyskanie propozycji dań spełniających ograniczenia czasowe.

seasonal_recipes - Narzędzie realizuje wyszukiwanie przepisów opartych na sezonowości składników. Wykorzystuje relacje pomiędzy encjami `Ingredient` i `Season` w grafie, umożliwiając filtrowanie przepisów według pory roku oraz opcjonalnych kryteriów dietetycznych i czasowych. Sezon jest normalizowany po stronie systemu do jednej z predefiniowanych wartości.

Narzędzia RAG (Cel: semantyczne wyszukiwanie oparte o embeddingi chunków - Classic RAG):

rag_search - Narzędzie realizuje klasyczne wyszukiwanie semantyczne w warstwie RAG na poziomie fragmentów tekstu (chunków). Zapytanie użytkownika jest embedowane do przestrzeni wektorowej, a następnie porównywane z embeddingami zapisanymi w indeksie wektorowym Neo4j. Wynikiem są fragmenty dokumentów o najwyższym podobieństwie semantycznym.

rag_search_recipes - Rozszerzona wersja wyszukiwania RAG, w której wyniki wyszukiwania na poziomie chunków są agregowane do poziomu przepisów. Dla każdego przepisu wybierane są najbardziej relewantne fragmenty, a następnie przepisy są rankowane na podstawie najlepszego dopasowania semantycznego. Narzędzie umożliwia porównywalność wyników z wyszukiwaniem grafowym.

Narzędzia użytkownika:

create_user - Narzędzie służy do tworzenia nowego użytkownika systemu. Umożliwia zapis podstawowych danych użytkownika oraz opcjonalnych profili dietetycznych, które następnie wykorzystywane są w procesach wyszukiwania i rekomendacji.

add_pantry_items - Narzędzie umożliwia dodawanie produktów do spiżarni użytkownika. W ramach jego działania uruchamiany jest Matching Engine, który dopasowuje nazwy składników do istniejących encji w grafie lub inicjuje utworzenie nowych węzłów. Produkty zapisywane są wraz z ilością, jednostką oraz opcjonalną datą ważności.

show_pantry - Narzędzie zwraca aktualną zawartość spiżarni użytkownika, obejmującą listę produktów wraz z ich ilością, jednostką oraz datą ważności. Dane są

sortowane w sposób umożliwiający szybkie zidentyfikowanie produktów zbliżających się do przeterminowania.

get_missing_ingredients - Narzędzie generuje listę brakujących składników wymaganych do przygotowania wybranego przepisu. Porównuje ono składniki przepisu z zawartością spiżarni użytkownika i zwraca zarówno składniki posiadane, jak i brakujące, wraz z informacją o wymaganych ilościach.

2.3.5. Frontend Layer

Warstwa frontendowa systemu **Food Waste Manager** została zrealizowana jako aplikacja webowa oparta o framework **Streamlit**.

Pełni ona rolę interaktywnego interfejsu użytkownika oraz punktu wejścia do komunikacji z systemem opartym o modele językowe (LLM) oraz graf wiedzy.

Frontend odpowiada za:

- interakcję użytkownika z systemem w formie konwersacyjnej (chat),
- zarządzanie lokalnym stanem rozmów,
- wprowadzanie danych wejściowych, takich jak lista produktów oraz zapytania użytkownika,
- prezentację odpowiedzi generowanych przez inteligentną warstwę systemu.

Architektura frontendu

Frontend został zaprojektowany jako aplikacja Streamlit uruchamiana w kontenerze **Docker**, z wykorzystaniem **docker-compose** do zapewnienia spójnego środowiska uruchomieniowego.

Cała logika interfejsu użytkownika, obsługa stanu sesji oraz prosta warstwa przechowywania danych zostały zawarte w jednym module aplikacyjnym **app.py**.

Kluczowe elementy architektury frontendowej:

- **Streamlit UI** – odpowiedzialny za renderowanie widoków, formularzy oraz elementów interaktywnych,
- **Session State** – mechanizm Streamlit służący do przechowywania aktualnego stanu sesji użytkownika,
- **JSON-based storage** – lekka, lokalna warstwa przechowywania danych użytkownika,
- **Docker + docker-compose** – zapewniające powtarzalne i izolowane środowisko uruchomieniowe.

Funkcjonalności interfejsu użytkownika

Frontend aplikacji został podzielony na kilka logicznych sekcji, dostępnych poprzez boczne menu nawigacyjne.

1. Home

Sekcja startowa aplikacji, której celem jest wprowadzenie użytkownika w funkcjonalności systemu oraz jego główne założenia.

2. Chat

Główna część aplikacji, umożliwiająca:

- interakcję z systemem w formie rozmowy tekstowej,
- obsługę wielu sesji rozmów (historia czatów),
- rozpoczynanie nowych konwersacji,
- sortowanie rozmów według czasu ich utworzenia.

Historia rozmów jest przechowywana lokalnie w pliku `chats.json`, natomiast bieżąca sesja rozmowy zarządzana jest przy użyciu mechanizmu `st.session_state`.

3. Add Ingredients

Moduł umożliwiający użytkownikowi:

- ręczne dodawanie produktów spożywczych,
- zarządzanie listą posiadanych składników.

Dane dotyczące składników przechowywane są w pliku `ingredients.json` i stanowią dodatkowe wejście kontekstowe wykorzystywane podczas przetwarzania zapytań w module czatu.

W obecnej wersji systemu moduł **Add Ingredients** pełni funkcję wspierającą i demonstracyjną, stanowiąc podstawę do dalszego rozwoju w kierunku automatycznej synchronizacji produktów, integracji z bazą danych lub rozszerzenia logiki rekomendacyjnej systemu.

4. Feedback

Sekcja umożliwiająca zbieranie opinii użytkowników na temat działania systemu.

Moduł ten został zaprojektowany z myślą o przyszłej rozbudowie o funkcje analityczne i ewaluacyjne.

W aktualnej wersji aplikacji sekcja **Feedback** pełni rolę przygotowanego punktu integracyjnego pod przyszłe funkcjonalności analityczne i monitoringowe.

Zarządzanie stanem i danymi

Frontend wykorzystuje:

- **Streamlit Session State** do zarządzania:
 - aktywną rozmową,
 - listą dostępnych czatów,
 - inicjalizacją sesji użytkownika,
- **pliki JSON** jako lekką warstwę persistence:
 - `chats.json` – przechowujący historię rozmów,
 - `ingredients.json` – przechowujący listę produktów użytkownika.

Takie podejście umożliwia szybkie prototypowanie oraz testowanie systemu bez konieczności integracji z zewnętrzną bazą danych po stronie frontendu.

Integracja z backendem i warstwą inteligentną

Frontend pełni rolę warstwy prezentacyjnej oraz wejściowej systemu:

- zbiera zapytania użytkownika,
- przekazuje je do warstwy przetwarzania (LLM oraz graf wiedzy),
- prezentuje wygenerowane odpowiedzi w czytelnej, konwersacyjnej formie.

Architektura frontendu została zaprojektowana w sposób umożliwiający:

- łatwe podpięcie dedykowanego API backendowego,
- rozdzielenie warstwy prezentacyjnej i obliczeniowej w kolejnych iteracjach systemu.

Technologie

- Python
- Streamlit
- Docker
- docker-compose
- JSON

3. Knowledge Graph

3.1. Schemat grafu (encje i relacje).

Schemat grafu wiedzy został zaprojektowany jako graf właściwości, w którym podstawowymi węzłami są: Recipe (przepis), Ingredient (składnik), Chunk (fragment tekstu / krok przepisu),

Document (źródło / strona / książka kucharska), User, PantryItem (spizarnia użytkownika), Dietary Profile, Cuisine, Tag oraz Season.

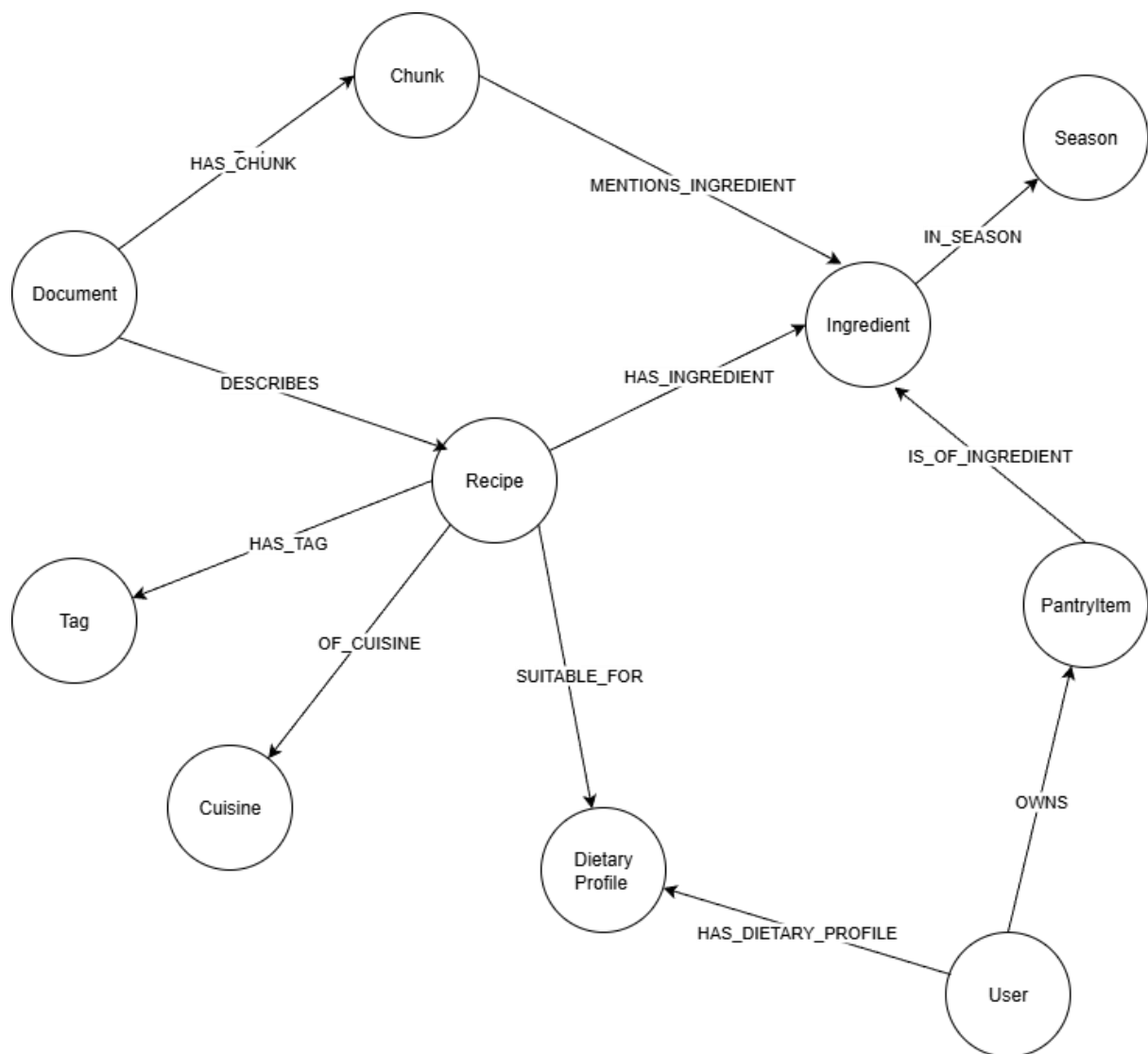
Centralnym węzłem jest Recipe, który łączy w sobie większość istotnych relacji. Przepis:

- **HAS_INGREDIENT** → Ingredient
- **HAS_CHUNK** → Chunk (poszczególne kroki/instrukcje),
- **HAS_TAG** → Tag (np. „pieczone”, „deser”, „łatwe”),
- **OF_CUISINE** → Cuisine (kuchnia włoska, tajska, polska itp.),
- **SUITABLE_FOR** → Dietary Profile (odpowiedni dla diety niskowęglowodanowej, bezglutenowej, keto itp.).

Składniki (Ingredient) są powiązane z:

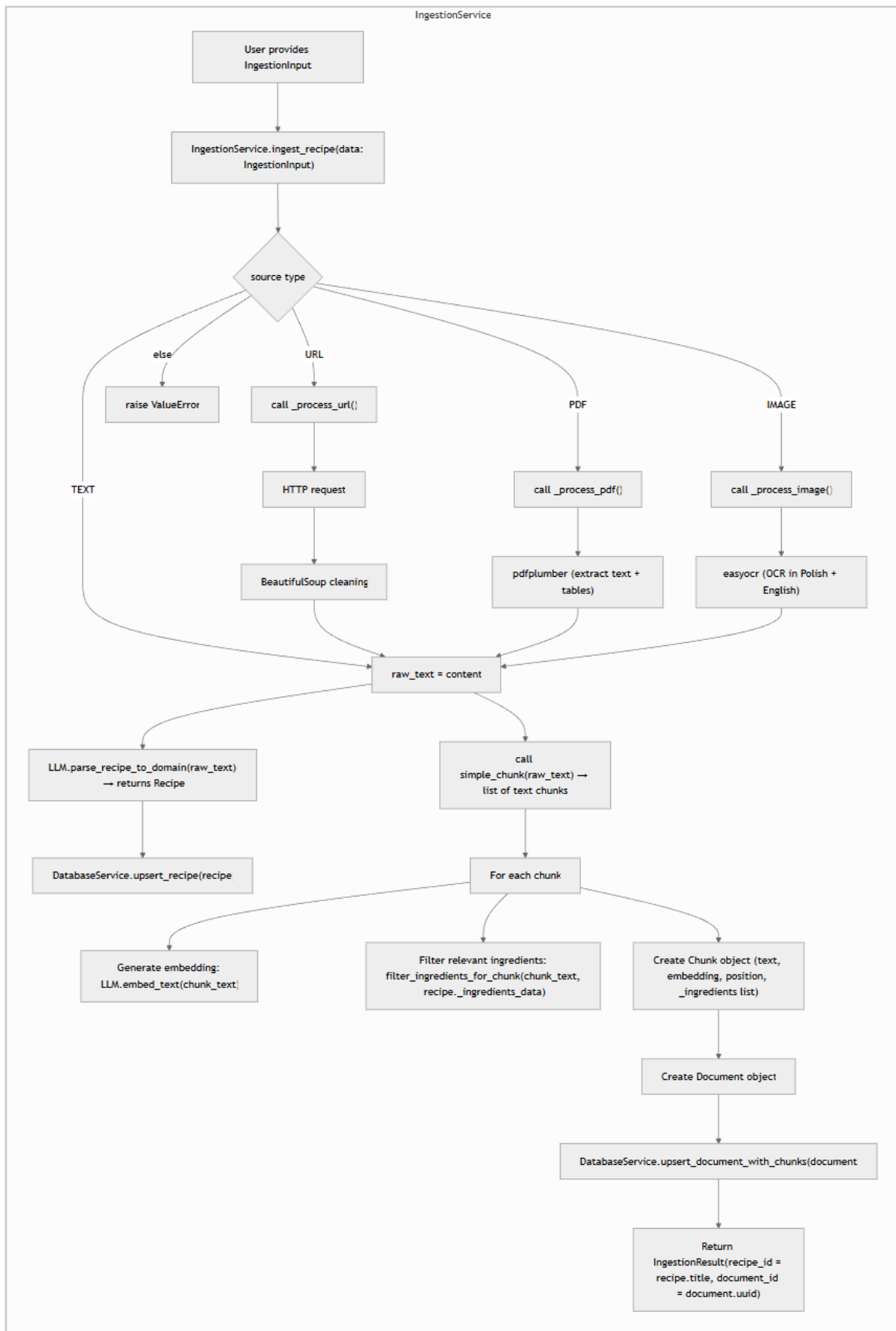
- **IN_SEASON** → Season (porą roku, w której składnik jest najbardziej dostępny),
- **IS_OF_INGREDIENT** ← PantryItem (konkretna rzecz w spizarni użytkownika),
- **MENTIONS_INGREDIENT** ← Chunk (w którym składnik został wspomniany w tekście).

Użytkownik może posiadać jedną lub więcej PantryItem opisującego ten sam produkt, ponieważ na tym węźle są właściwości takie jak ilość i data ważności (możemy mieć 5 jajek z datą ważności do jutra i 10 jajek z datą ważności do końca tygodnia).



3.2. Jak przetwarzano dane wejściowe:

Kluczowym modulem przetwarzającym dane wejściowe jest IngestionService (opisany w 2. Opis architektury systemu). Model działania przedstawiono poniżej.



3.3. Metody walidacji danych i zapewnienia jakości ekstrakcji.

- LLM otrzymuje bardzo szczegółowy, wielostronicowy system prompt z twardymi regułami (never null, zawsze inferować rozsądne wartości, ścisła normalizacja nazw składników, jednostek, kategorii, sezonowości)
- Zastosowano reguły biznesowe „ostatniej szansy” dla brakujących ilości: 1 szt., 1 łyżeczka, 1 łyżka, 1 ząbek itp.. Takie podejście eliminuje puste pola
- Wymuszona normalizacja nazw składników (usuwanie przymiotników, form przygotowania, wielkości) → „large whole chicken” → „chicken”
- Sezonowość przypisywana wg ścisłych reguł kalendarzowych + domyślnie „year-round” przy braku wyraźnych przesłanek
- Automatyczne wnioskowanie profili dietetycznych (vegan / vegetarian / gluten-free / dairy-free) wyłącznie na podstawie listy składników – bez możliwości pominięcia
- Wyjście strukturyzowane za pomocą LangChain with_structured_output(RecipeSchema). LLM nie może zwrócić niepoprawnego formatu JSON
- W warstwie aplikacji sprawdzana jest zgodność wymiarowości embeddingów z oczekiwaną wartością (stała walidacja przy każdym embedowaniu)
- Dopasowywanie nowych składników do istniejących w bazie odbywa się poprzez dedykowany prompt semantycznego dopasowania + zwracanie confidence score
- Wszystkie brakujące / niepewne pola są wypełniane najbardziej prawdopodobną wartością zamiast null / pustego stringa

Te mechanizmy łącznie znacząco zmniejszają liczbę niepoprawnych rekordów i niespójności w grafie wiedzy przy wstawianiu nowych rekordów.

4. Algorytm dopasowania danych z różnych źródeł (Matching Algorithm)

W systemie zastosowano algorytm dopasowania danych odpowiedzialny za **scalanie i normalizację składników (Ingredient)** pochodzących z różnych źródeł wejściowych. Algorytm ten jest wykorzystywany wyłącznie w kontekście **dodawania produktów do spiżarni użytkownika** i ma na celu zapobieganie duplikowaniu encji w grafowej bazie danych Neo4j.

Algorytm nie realizuje klasycznego rankingu wielu kandydatów ani porównywania embeddingów nazw składników. Zamiast tego wykorzystuje **LLM jako mechanizm decyzyjny (LLM-as-a-Judge)** oraz **twardy próg akceptacji**, który rozstrzyga, czy dany składnik powinien zostać połączony z istniejącą encją w grafie, czy zapisany jako nowa encja.

4.1. Jak działa algorytm dopasowania.

Algorytm uruchamiany jest w metodzie `IngestionService.ingest_pantry_items_for_user` w momencie dodawania produktów do spiżarni.

1. Pobranie zbioru referencyjnego

Z bazy danych pobierana jest pełna lista istniejących encji `Ingredient`. Lista ta stanowi jedyny zbiór kandydatów dopasowania i pełni rolę kanonicznego słownika składników systemu.

2. Przygotowanie danych wejściowych

Nowe produkty przekazane przez użytkownika są mapowane do listy obiektów zawierających nazwę składnika oraz indeks wejściowy. Indeks ten umożliwia jednoznaczne przypisanie wyniku dopasowania do konkretnego elementu wejścia.

3. Semantyczne dopasowanie przez model językowy

Jeżeli w bazie istnieją jakiekolwiek składniki, system wywołuje metodę `LLMClient.match_ingredients_to_existing`, przekazując:

- listę nazw istniejących składników,
- listę nowych nazw składników użytkownika.

4. Model językowy zwraca dla każdego elementu wejściowego:

- `matched_name` – nazwę najlepiej dopasowanego istniejącego składnika lub `null`,
- `confidence` – liczbową ocenę pewności dopasowania.

5. Decyzja progowa (gating)

Dla każdego elementu wejściowego stosowany jest próg `_match_threshold = 0.7`:

- jeżeli `matched_name` jest ustawione oraz `confidence ≥ 0.7`, system uznaje dopasowanie za poprawne i wykorzystuje istniejącą encję `Ingredient`,
- w przeciwnym przypadku tworzona jest nowa encja `Ingredient` z nazwą wejściową użytkownika.

6. Zapis w grafie

Dla każdego produktu tworzona jest encja `PantryItem`, która następnie łączona jest:

- z użytkownikiem (`User → PantryItem`),
- z wybraną lub nowo utworzoną encją `Ingredient` (`PantryItem → Ingredient`).

W ten sposób wynik dopasowania zostaje trwale zapisany w strukturze grafowej.

4.2. Jak uwzględnia różne czynniki: ingredients match, doświadczenie, dostępność.

Algorytm dopasowania w tej aplikacji nie wykorzystuje wielu niezależnych wag ani ręcznie definiowanych cech. Wszystkie czynniki są uwzględniane pośrednio przez semantyczną ocenę modelu językowego oraz mechanizm progu decyzyjnego.

- **Różnice językowe i synonimy**
Dopasowanie odbywa się na poziomie znaczenia nazwy składnika, a nie identyczności ciągów znaków. Pozwala to na łączenie składników zapisanych w różnych językach lub wariantach leksykalnych (np. różne formy tej samej nazwy).
- **Różne formy zapisu i błędy wejściowe**
Algorytm toleruje odmiany gramatyczne, skróty oraz potencjalne błędy pochodzące z OCR lub ekstrakcji tekstu, ponieważ decyzja opiera się na semantycznym osądzie modelu językowego.
- **Stabilność ontologii składników**
Istniejące encje Ingredient w bazie pełnią rolę stabilnego punktu odniesienia. Wraz z rozwojem bazy rośnie prawdopodobieństwo poprawnego dopasowania nowych danych bez tworzenia zbędnych duplikatów.
- **Bezpieczeństwo dopasowania**
W przypadku niepewnego dopasowania algorytm preferuje utworzenie nowej encji zamiast ryzykownego scalenia, co chroni graf przed błędnymi połączeniami semantycznymi.

4.3. Wyjaśnienie sposobu punktacji / rankingowania kandydatów

W aktualnej implementacji algorytm **nie wykonuje jawnego rankingu wielu kandydatów** ani sortowania wyników po podobieństwie. Mechanizm punktacji i wyboru ma następującą postać:

Punktacja

Jedyną miarą jakości dopasowania jest wartość confidence zwracana przez model językowy dla pojedynczego najlepszego dopasowania.

Ranking

Ranking odbywa się wewnątrznie po stronie modelu językowego, który wybiera jeden najlepszy kandydat (matched_name) dla każdej nazwy wejściowej. System nie otrzymuje listy top-N dopasowań.

Decyzja końcowa

Ostateczna decyzja jest binarna i oparta na progu:

- $\text{confidence} \geq 0.7 \rightarrow$ scalenie z istniejącym składnikiem,

- confidence < 0.7 lub brak dopasowania → utworzenie nowego składnika.

Takie podejście upraszcza logikę systemu, eliminuje konieczność ręcznego strojenia wag oraz zapewnia pełną kontrolę nad momentem, w którym dane są scalane w grafie.

5. Business Intelligence / Zaawansowane zapytania

System **Food Waste Manager** wspiera zaawansowane zapytania analityczne i decyzyjne, wykorzystując połączenie grafowej bazy danych **Neo4j** oraz przetwarzania języka naturalnego przez model językowy (LLM).

Zapytania użytkownika są analizowane semantycznie, a następnie mapowane na operacje wyszukiwania, filtrowania i reasoning'u w grafie wiedzy.

Architektura systemu umożliwia obsługę zarówno zapytań prostych, jak i złożonych scenariuszy decyzyjnych, w których brane są pod uwagę kontekst rozmowy, dostępność składników oraz dodatkowe ograniczenia logiczne.

5.1. Lista obsługiwanych typów zapytań

5.1.1. Zapytania informacyjne i eksploracyjne

Zapytania mające na celu eksplorację dostępnej wiedzy w systemie, w tym przepisów, składników oraz ich relacji.

Przykłady:

- Jakie przepisy są dostępne w systemie?
- Jakie składniki są wykorzystywane w przepisach?
- Jakie dania należą do kuchni włoskiej?

5.1.2. Liczenie (Counting)

Zapytania polegające na obliczaniu liczby obiektów spełniających określone warunki w grafie wiedzy.

Przykłady:

- Ile przepisów mogę ugotować z produktów, które mam?
- Ile przepisów spełnia dietę wegetariańską?
- Ile potraw można przygotować w mniej niż 30 minut?

5.1.3. Filtrowanie (Filtering)

Selekcja przepisów na podstawie jednego lub wielu kryteriów logicznych i kontekstowych.

Obsługiwane kryteria obejmują m.in.:

- dostępność składników,
- czas przygotowania,
- typ kuchni,
- ograniczenia dietetyczne,
- preferencje użytkownika,
- priorytet wykorzystania wybranych produktów.

Przykłady:

- Co mogę ugotować z produktów, które mam?
- Jakie przepisy są wegetariańskie i szybkie?
- Co mogę ugotować z pomidorów i makaronu?

5.1.4. Agregacja (Aggregation)

Zapytania wymagające zestawienia informacji pochodzących z wielu węzłów grafu oraz ich relacji.

Przykłady:

- Jakie składniki najczęściej występują w przepisach?
- Które przepisy wykorzystują największą liczbę dostępnych składników?
- Jakie produkty pojawiają się w wielu przepisach jednocześnie?

5.1.5. Reasoning i zapytania wieloetapowe (Multi-hop reasoning)

Złożone zapytania wymagające przechodzenia przez wiele relacji w grafie wiedzy oraz łączenia informacji z różnych kontekstów.

System wykorzystuje:

- relacje **przepis** → **składniki** → **dostępność**,
- kontekst rozmowy przechowywany w sesji czatu,
- dodatkowe warunki logiczne zadane w języku naturalnym.

Przykłady:

- Jakie dania mogę ugotować z produktów, które mam, jeśli chcę zużyć konkretne składniki?
- Jakie przepisy spełniają jednocześnie warunki dietetyczne i czasowe?

5.1.6. Zapytania temporalne (czasowe)

Zapytania uwzględniające czas jako istotny czynnik decyzyjny.

Przykłady:

- Co mogę ugotować w 30 minut?
- Jakie przepisy są najszybsze do przygotowania?

5.1.7. Złożone scenariusze decyzyjne (What-if)

Najbardziej zaawansowany typ zapytań, łączący wiele kryteriów i ograniczeń, testowany w formie zapytań konwersacyjnych.

System umożliwia:

- łączenie wielu warunków w jednym zapytaniu,
- analizę kompromisów pomiędzy czasem, dostępnością składników i preferencjami,
- generowanie odpowiedzi opisowych wraz z uzasadnieniem.

Przykład:

Co mogę ugotować szybko, jeśli mam tylko kilka składników i chcę ich użyć jak najwięcej?

5.2. Przykłady zapytań i odpowiedzi systemu

Rodzaj zapytania	Zapytanie użytkownika	Odpowiedź systemu
Zapytania czasowe i filtrowanie	Co mogę ugotować w 30 minut?	Lista przepisów, których czas przygotowania nie przekracza 30 minut.
Zapytania czasowe i filtrowanie	Pokaż przepisy, które można ugotować w 30 minut. Limit 20.	Lista maksymalnie 20 przepisów spełniających ograniczenie czasowe.
Zarządzanie spiżarnią (pantry)	Dodaj do mojej spiżarni: pomidory 3 szt (data 2026-01-16), cukinia 2 szt (data 2026-01-15), makaron 500 g (data brak), jajka 10 szt (data 2026-01-20).	Potwierdzenie dodania produktów do spiżarni użytkownika wraz z ich ilością i datą przydatności.
Zarządzanie spiżarnią (pantry)	Pokaż spiżarnię	Lista aktualnie posiadanych produktów wraz z ilością i datą przydatności.
Rekomendacje na podstawie dostępnych produktów	Co mogę ugotować z produktów, które mam w domu?	Lista przepisów możliwych do przygotowania na podstawie produktów znajdujących się w spiżarni użytkownika.
Rekomendacje na podstawie dostępnych produktów	Co mogę ugotować z: pomidor, cukinia, makaron. Pokaż maksymalnie 5 pozycji.	Lista do 5 przepisów wykorzystujących wskazane składniki.
Rekomendacje na podstawie dostępnych produktów	Co mogę ugotować z produktów, którym kończy się data przydatności w	Propozycje przepisów zoptymalizowanych pod wykorzystanie produktów z

	ciągu 30 dni? Pokaż propozycje.	krótką datą przydatności.
Zarządzanie przepisami	Dodaj przepis: "Makaron z cukinią i pomidorami". Składniki: makaron 200g, cukinia 1 szt, pomidory 2 szt, czosnek 1 ząbek, oliwa. Czas: 20 minut. Kroki: ugotuj makaron, podsmaż cukinię i czosnek, dodaj pomidory, połącz.	Potwierdzenie dodania przepisu do grafu wiedzy wraz z ekstrakcją składników, czasu przygotowania oraz kroków.
Zarządzanie przepisami	Dodaj przepis z linku.	Pobranie treści przepisu z podanego adresu URL, jego analiza i zapis do bazy danych.
Zarządzanie przepisami	Dodaj przepis z PDF.	Ekstrakcja treści przepisu z pliku PDF oraz zapis ustrukturyzowanych danych w grafie wiedzy.
Zarządzanie przepisami	Dodaj przepis ze zdjęcia.	Analiza obrazu, rozpoznanie treści przepisu i zapis do systemu.
Listy zakupów	Zrób listę zakupów brakujących produktów do przepisu o nazwie <i>Creole Green Beans</i>	Lista składników wymaganych do przygotowania danego przepisu, które nie znajdują się w spiżarni użytkownika.
Zapytania sezonowe i kontekstowe	Jakie dania można ugotować, w których można wykorzystać sezonowe warzywa (zimowe)?	Lista przepisów wykorzystujących produkty sezonowe charakterystyczne dla okresu zimowego.
Zapytania sezonowe i kontekstowe	Jakie dania można ugotować, w których można wykorzystać sezonowe produkty (zimowe)?	Rekomendacje przepisów bazujących na sezonowych produktach zimowych.
Zapytania dietetyczne (GraphRAG)	Co mogę ugotować dla osoby z celiakią z produktów które mam?	Lista przepisów bezglutenowych, dopasowanych do produktów dostępnych w spiżarni użytkownika.

6. Wyniki eksperymentów / metryki

W celu oceny skuteczności zaprojektowanego algorytmu dopasowania przeprowadzono eksperyment porównawczy pomiędzy dwoma strategiami wyszukiwania zaimplementowanymi w systemie: deterministycznym wyszukiwaniem grafowym (Graph-based retrieval) oraz klasycznym wyszukiwaniem semantycznym opartym o embeddingi (Traditional RAG). Ewaluacja została wykonana przy użyciu dedykowanego skryptu `eval_rag_vs_graph.py` oraz zestawu zapytań testowych zapisanych w pliku `rag_vs_graph.json`.

Eksperyment obejmował analizę jakości dopasowania wyników, skuteczności rankingowania oraz czasu odpowiedzi systemu dla obu podejść.

6.1. Dokładność dopasowania (matching accuracy)

Dokładność dopasowania została oceniona przy użyciu metryk rankingowych obliczonych dla listy Top-5 wyników (@K=5). Jako zbiór referencyjny (ground truth) wykorzystano ręcznie zdefiniowane identyfikatory relewantnych przepisów dla danego zapytania.

Dla zapytania testowego opartego na składnikach ['kurczak', 'ryż'] zbiór referencyjny zawierał dwa relewantne przepisy. Wyszukiwanie grafowe zwróciło oba relewantne elementy w pierwszej piątce wyników, podczas gdy wariant RAG nie zwrócił żadnego z nich.

Wyniki metryk rankingowych przedstawiają się następująco:

- **Graph-based search**
 - Precision@5: 0.4
 - Recall@5: 1.0
 - MRR@5: 1.0
 - nDCG@5: 1.0
- **Traditional RAG**
 - Precision@5: 0.0
 - Recall@5: 0.0
 - MRR@5: 0.0
 - nDCG@5: 0.0

Oznacza to, że wyszukiwanie grafowe poprawnie zidentyfikowało wszystkie relewantne przepisy i umieściło je na najwyższych pozycjach listy wyników, natomiast wyszukiwanie semantyczne nie było w stanie odnaleźć żadnego z oczekiwanych rezultatów.

6.2. Czas odpowiedzi na zapytania.

Czas odpowiedzi systemu został zmierzony osobno dla obu strategii wyszukiwania. Analizowane były statystyki p50, p95 oraz średni czas odpowiedzi.

- **Graph-based search**

- p50: 1658.46 ms
- p95: 1658.46 ms
- średnia: 1658.46 ms

- **Traditional RAG**

- p50: 1620.51 ms
- p95: 1620.51 ms
- średnia: 1620.51 ms

Uzyskane wyniki wskazują, że czasy odpowiedzi obu podejść są porównywalne i mieszczą się w podobnym przedziale czasowym. Różnice pomiędzy Graph-based search a RAG są nieznaczne i nie mają istotnego wpływu na ogólną responsywność systemu.

6.3. Efektywność grafu (średni czas przeszukiwania Cypher).

Efektywność wyszukiwania grafowego została oceniona na podstawie pomiarów czasu odpowiedzi dla zapytań realizowanych przy użyciu języka Cypher w bazie danych Neo4j. Pomiar obejmował całkowity czas obsługi zapytania, w tym wykonanie zapytania grafowego, agregację wyników oraz przygotowanie odpowiedzi dla warstwy aplikacyjnej. Do analizy wykorzystano statystyki p50, p95 oraz średni czas odpowiedzi.

Dla wyszukiwania grafowego uzyskano następujące wyniki:

- p50: 1658.46 ms
- p95: 1658.46 ms
- średni czas odpowiedzi: 1658.46 ms

Dla porównania, w przypadku wyszukiwania semantycznego opartego na embeddingach (Traditional RAG) uzyskano:

- p50: 1620.51 ms
- p95: 1620.51 ms
- średni czas odpowiedzi: 1620.51 ms

Uzyskane wyniki wskazują, że czas odpowiedzi zapytań grafowych jest zbliżony do czasu odpowiedzi wyszukiwania wektorowego, a różnica pomiędzy średnimi wartościami wynosi około 38 ms. Różnica ta jest niewielka i nie wpływa istotnie na postrzeganą responsywność systemu.

Należy podkreślić, że zapytania grafowe obejmują wieloetapowe operacje, takie jak filtrowanie przepisów po składnikach, obliczanie liczby dopasowanych składników

(matched_count) oraz sortowanie wyników według stopnia dopasowania. Pomimo tej złożoności czas odpowiedzi pozostaje stabilny i przewidywalny, co potwierdzają identyczne wartości p50 i p95.

Na podstawie przeprowadzonych pomiarów można stwierdzić, że zastosowanie grafowej reprezentacji wiedzy oraz zapytań Cypher nie wprowadza istotnego narzutu czasowego w porównaniu do podejścia opartego wyłącznie na wyszukiwaniu wektorowym. Jednocześnie, w zestawieniu z wynikami jakościowymi przedstawionymi w poprzednich podrozdziałach, wyszukiwanie grafowe zapewnia znacznie wyższą skuteczność dopasowania, co czyni je efektywnym rozwiązaniem w kontekście projektowanego systemu.

6.4. Porównanie GraphRAG vs. tradycyjny RAG:

6.4.1. Porównanie wydajności

Pod względem wydajności czasowej oba podejścia osiągają zbliżone wyniki. Traditional RAG jest minimalnie szybszy, jednak różnica ta nie jest istotna w kontekście całkowitego czasu odpowiedzi systemu.

6.4.2. Dokładność odpowiedzi na złożone zapytania

Dla zapytań opartych na konkretnych składnikach Graph-based search wykazuje zdecydowanie wyższą skuteczność. Jawne relacje pomiędzy encjami Recipe i Ingredient umożliwiają precyzyjne dopasowanie wyników do zapytania użytkownika. W przeciwieństwie do tego, wariant RAG oparty wyłącznie na podobieństwie semantycznym tekstu nie był w stanie poprawnie zidentyfikować relewantnych przepisów.

6.4.3. Czas odpowiedzi

Czas odpowiedzi obu podejść pozostaje na zbliżonym poziomie, co oznacza, że wybór strategii grafowej nie wiąże się z pogorszeniem responsywności systemu.

6.4.4. Wybrane metryki ewaluacyjne

Do porównania wykorzystano standardowe metryki rankingowe:

- Precision@K,
- Recall@K,
- Mean Reciprocal Rank (MRR),
- Normalized Discounted Cumulative Gain (nDCG).

Zestawienie tych metryk jednoznacznie wskazuje na przewagę wyszukiwania grafowego w kontekście zapytań strukturalnych opartych na składnikach, przy zachowaniu porównywalnych czasów odpowiedzi.

Wnioski z testu:

Przeprowadzone eksperymenty pokazują, że w kontekście tego projektu Graph-based retrieval stanowi skuteczniejszą metodę dopasowania niż klasyczny RAG oparty wyłącznie na embeddingach. Wyszukiwanie grafowe zapewnia pełną trafność wyników dla analizowanego zapytania, wysoką jakość rankingu oraz stabilny czas odpowiedzi. Wyniki te uzasadniają wybór grafowej reprezentacji wiedzy jako podstawowego mechanizmu dopasowania w systemie.

7. Wnioski i rekomendacje

7.1. Co działało najlepiej w systemie GraphRAG?

Najlepsze rezultaty w systemie uzyskano przy zastosowaniu wyszukiwania grafowego opartego na jawnych relacjach pomiędzy encjami Recipe i Ingredient. Wyszukiwanie to wykazało wysoką skuteczność zarówno pod względem trafności wyników, jak i jakości ich rankingu.

Dla analizowanego zapytania testowego Graph-based search poprawnie zidentyfikował wszystkie relewantne przepisy i umieścił je w pierwszej piątce wyników. Potwierdzają to wartości metryk rankingowych: Recall@5 = 1.0 oraz MRR@5 = 1.0, co oznacza, że każdy relewantny element został odnaleziony, a przynajmniej jeden z nich znalazł się na pierwszej pozycji listy wyników. Wysoka wartość nDCG@5 = 1.0 wskazuje również na poprawną kolejność wyników względem ich istotności.

Istotnym aspektem jest także stabilność czasowa wyszukiwania grafowego. Pomimo wykorzystania wieloetapowych zapytań Cypher, obejmujących filtrowanie po składnikach, liczenie dopasowań oraz sortowanie wyników, czas odpowiedzi pozostał porównywalny z podejściem opartym na embeddingach. Średni czas odpowiedzi na poziomie około 1.66 s oraz brak istotnych różnic pomiędzy p50 i p95 wskazują na przewidywalne i stabilne zachowanie systemu.

Na podstawie uzyskanych wyników można stwierdzić, że GraphRAG w obecnej implementacji najlepiej sprawdza się w przypadku zapytań strukturalnych, w których kluczowe znaczenie ma jednoznaczne dopasowanie składników do przepisów. Jawna reprezentacja wiedzy w postaci grafu umożliwia precyzyjne i deterministyczne wyszukiwanie, co bezpośrednio przekłada się na wysoką jakość wyników.

7.2. Gdzie tradycyjny RAG był niewystarczający?

Tradycyjne wyszukiwanie RAG oparte wyłącznie na podobieństwie embeddingów fragmentów tekstu okazało się niewystarczające w kontekście zapytań wymagających precyzyjnego dopasowania składników. Dla tego samego zapytania testowego wariant RAG nie zwrócił żadnego relewantnego przepisu w pierwszej piątce wyników, co znalazło

odzwierciedlenie w zerowych wartościach wszystkich analizowanych metryk rankingowych (Precision@5, Recall@5, MRR@5 oraz nDCG@5).

Wyniki te wskazują, że podobieństwo semantyczne fragmentów tekstu nie gwarantuje poprawnego dopasowania na poziomie encji domenowych, takich jak składniki przepisu. W analizowanym przypadku embeddingi chunków nie były wystarczająco skorelowane z listą składników wejściowych, co doprowadziło do braku trafnych wyników, mimo że relewantne przepisy istniały w bazie danych.

Należy podkreślić, że słabość tradycyjnego RAG nie wynikała z ograniczeń wydajnościowych. Czasy odpowiedzi dla wyszukiwania wektorowego były porównywalne, a nawet nieznacznie niższe niż dla wyszukiwania grafowego. Oznacza to, że problem dotyczył wyłącznie jakości dopasowania, a nie kosztu obliczeniowego.

Na podstawie uzyskanych wyników można wnioskować, że w obecnej architekturze tradycyjny RAG nie jest odpowiednim mechanizmem do realizacji zapytań opartych na precyzyjnych kryteriach strukturalnych, takich jak lista wymaganych składników. Jego zastosowanie w systemie powinno być ograniczone do scenariuszy, w których zapytania mają charakter opisowy lub narracyjny, a nie deterministyczny.

7.3. Ewentualne ograniczenia i kierunki rozwoju?

System wykorzystuje pojedyncze wywołanie LLM do wyboru narzędzia, co ogranicza możliwość obsługi złożonych, wieloetapowych zapytań wymagających kontekstu poprzednich interakcji. Brakuje mechanizmu zarządzania historią konwersacji oraz możliwości wielokrotnego wywoływania narzędzi w ramach jednego zapytania użytkownika. Ekstrakcja tekstu z PDF i obrazów (OCR) opiera się na bibliotekach zewnętrznych (pdfplumber, easyocr) bez zaawansowanej walidacji jakości rozpoznawania. System nie implementuje mechanizmów rate-limitingu ani zarządzania limitami API Azure OpenAI, co może prowadzić do problemów przy intensywnym użyciu. Dopasowywanie składników do bazy (match_ingredients_to_existing) wykorzystuje LLM bez cachingu, co generuje dodatkowe koszty i opóźnienia przy każdej operacji dodawania produktów do spiżarni.

7.4. Czego nie udało się zrobić? Co można było zrobić lepiej?

Wszystkie wymagania, zdefiniowane w dokumencie wstępnym zostały zaimplementowane, przetestowane i wdrożone, natomiast jest dużo rzeczy, które można było zrobić lepiej.

- **Architektura Agenta i Zarządzanie Kontekstem**

W obecnej wersji agent (AgentService) działa bez stanowo (stateless) – każde zapytanie jest traktowane jako nowa, izolowana interakcja.

Co poprawić: Wdrożyć klasę zarządzającą historią konwersacji (np. ChatHistory w LangChain), aby agent „pamiętał” poprzednie pytania (np. „Co z tego ugotuję?” po wcześniejszym dodaniu produktów).

Obecnie agent wybiera tylko jedno narzędzie na zapytanie (select_tool -> execute_tool). Warto wdrożyć pętlę ReAct (Reason-Act), która pozwoli agentowi

samodzielnie dopytać o brakujące parametry (np. „Jaka to dieta?”) przed wyszukaniem przepisu, zamiast zwracać błąd lub prośbę o doprecyzowanie.

Prompty są zaszyte w kodzie (prompts.py). Należy wydzielić je do plików konfiguracyjnych (np. YAML/JSON) lub użyć systemu zarządzania promptami (np. LangSmith), co ułatwi ich iterowanie bez zmiany kodu

- **Optymalizacja Kosztów i Wydajności LLM**

Metoda `match_ingredients_to_existing` wysyła wszystkie nazwy istniejących składników do LLM przy każdym dodawaniu produktu, co przy dużej bazie będzie generować ogromne koszty i opóźnienia.

Należałoby to zastąpić wyszukiwaniem wektorowym na poziomie bazy danych/biblioteki Python, a LLM używać tylko do ostatecznej weryfikacji trudnych przypadków.

- **Jakość ekstrakcji danych (OCR/PDF)**

Funkcje `_process_pdf` i `_process_image` polegają na prostych bibliotekach (pdfplumber, easyocr) i nie mają mechanizmu walidacji jakości (Quality Assurance). Należałoby dodać krok weryfikacji, czy wyekstrahowany tekst ma sens (np. czy zawiera słowa kluczowe jak "składniki", "instrukcje").

7.5. Jak system ma ograniczenia?

Agent obsługuje wyłącznie synchroniczne, pojedyncze zapytania bez kontekstu konwersacji, co uniemożliwia prowadzenie naturalnego dialogu wymagającego pamięci poprzednich interakcji. Promy systemowe są sztywno zakodowane w kodzie aplikacji, co utrudnia ich modyfikację i testowanie różnych wersji bez ponownego wdrażania systemu. System nie posiada mechanizmów obsługi błędów API Azure OpenAI - brak retry logic przy niedostępności usług LLM. Ekstrakcja przepisów z URL opiera się na prostym parsowaniu HTML bez dedykowanej obsługi popularnych serwisów kulinarnych, co może prowadzić do niskiej jakości wyników dla stron z nietypową strukturą.

8. Załączniki / dokumentacja techniczna

8.1. Instrukcje uruchomienia systemu (Docker, Neo4j, środowisko Python).

System może zostać uruchomiony zarówno lokalnie w środowisku Python, jak i w pełni konteneryzowanym środowisku Docker z wykorzystaniem Docker Compose.

8.1.1. Wymagania wstępne

- Python 3.12
- Docker oraz Docker Compose
- Dostęp do instancji **Azure OpenAI** (klucz API i endpoint)
- Uruchomiona instancja **Neo4j** (lokalnie lub w Dockerze)
- Przeglądarka internetowa (do interfejsu Streamlit)

8.1.2. Uruchomienie lokalne (Python)

1. Instalacja zależności:

```
pip install -r requirements.txt
```

2. Konfiguracja zmiennych środowiskowych:

W katalogu src/config należy utworzyć plik .env na podstawie env.example i uzupełnić go o:

- dane dostępowe do bazy Neo4j (DB_URI, DB_USER, DB_PASSWORD, DB_DATABASE),
- dane dostępowe do Azure OpenAI (AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_API_KEY, nazwy modeli),
- ścieżki do danych wejściowych (RAW_DATASET_PATH, DATASET_PATH)

4. Uruchomienie Neo4j lokalnie (np. poprzez oficjalny obraz Dockera) oraz ustawienie poprawnego DB_URI, DB_USER i DB_PASSWORD.

```
docker run -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/password \
  neo4j:latest
```

Po uruchomieniu należy upewnić się, że zmienne DB_URI, DB_USER oraz DB_PASSWORD w pliku .env są zgodne z konfiguracją instancji Neo4j.

Interfejs webowy Neo4j dostępny jest pod adresem: <http://localhost:7474>

5. Ingest pojedynczego pliku tekstowego:

```
python src/main.py
```

Skrypt inicjalizuje bazę danych Neo4j (tworzenie constraintów i indeksów), przetwarza plik tekstowy przepisu oraz zapisuje dane w grafie.

3. Inicjalizacja danych z pliku CSV:

```
python src/database/init_loader.py
```

Skrypt resetuje bazę danych i przetwarza zbiór przepisów z pliku CSV wskazanego w zmiennej RAW_DATASET_PATH.

4. Uruchomienie warstwy frontendowej (Streamlit)

Po uruchomieniu backendu oraz inicjalizacji danych możliwe jest uruchomienie interfejsu użytkownika:

```
streamlit run app.py
```

Aplikacja frontendowa dostępna jest domyślnie pod adresem: <http://localhost:8501>

Frontend umożliwia:

- interakcję z systemem w formie czatu,
- zarządzanie listą składników użytkownika,
- przeglądanie historii rozmów,
- testowanie zaawansowanych zapytań wspieranych przez backend.

8.1.3. Uruchomienie za pomocą Docker Compose

1. Upewnić się, że plik .env zawiera poprawne dane konfiguracyjne.
2. Uruchomienie systemu:

```
docker-compose up --build
```

Konfiguracja Docker Compose uruchamia:

- kontener Neo4j (porty 7474 – interfejs webowy oraz 7687 – Bolt),
- kontener aplikacyjny inicjalizujący dane i wykonujący pipeline ingestion.

Dane Neo4j są zapisywane w wolumenie lokalnym, co zapewnia ich trwałość pomiędzy restartami.

3. Zatrzymanie systemu:

```
docker-compose down
```

4. Zatrzymanie systemu wraz z usunięciem danych Neo4j:

```
docker-compose down -v
```